

Implement Access Control Measures

ACLs, Access Control Model, and User Access Levels — Healthcare IAM Home Lab

Prepared by Sabitha S Karanavar

1. Introduction

This report satisfies the project rubric item “Implement Access Control Measures” for the Healthcare IAM Operations Home Lab. It documents one Access Control List (ACL) configuration, one access control model in use, and one user access level, using the department shared folders that simulate the lab's line-of-business applications.

Environment note: the lab's domain controller runs Ubuntu Server with Samba4 providing Active Directory Domain Services. There is no NTFS on this host, so access control is enforced through Samba share permissions layered over POSIX ACLs on the underlying Linux filesystem — the same group-based concept as Windows NTFS + AD groups, implemented with different mechanisms.

2. Where Access Control Fits the RBAC Design

Week 1 of the project defined an RBAC matrix mapping each department role to a simulated application and an access level. Each simulated application is a Samba-shared folder on the Ubuntu Server; each role is an AD security group created with samba-tool. Access control, as documented in this report, is the enforcement layer that turns that matrix from a planning document into something the operating system actually honors: when a user connects to a share, Samba checks their AD group membership against the ACL on the underlying folder and allows or denies the connection accordingly.

3. ACL Configuration

The example below secures the shared folder for the “Nursing Charting” simulated application. Two AD groups are used: SG-App-NursingCharting-RW (read/write, assigned to the Nursing role) and SG-App-NursingCharting-RO (read-only, for a reporting/audit role). Both are mapped through Samba's idmap backend to Unix GIDs so that native POSIX ACL tools can reference them directly.

3.1 Create the Share Directory and Base Ownership

```
$ sudo mkdir -p /srv/samba/shares/nursing
$ sudo chown root:sambashare /srv/samba/shares/nursing
$ sudo chmod 0770 /srv/samba/shares/nursing
```

3.2 Apply the POSIX ACL

```
$ sudo setfacl -m g:"DAE\SG-App-NursingCharting-RW":rwx /srv/samba/shares/nursing
$ sudo setfacl -m g:"DAE\SG-App-NursingCharting-RO":r-x /srv/samba/shares/nursing
$ sudo setfacl -d -m g:"DAE\SG-App-NursingCharting-RW":rwx /srv/samba/shares/nursing
$ sudo setfacl -m o::--- /srv/samba/shares/nursing
```

The -d flag on the third command sets a default ACL, so new files and subfolders created inside the share automatically inherit the same group permissions. Verifying the result:

```
$ getfacl /srv/samba/shares/nursing
# file: srv/samba/shares/nursing
# owner: root
# group: sambashare
```

```

user::rwx
group::---
group:SG-App-NursingCharting-RW:rwx
group:SG-App-NursingCharting-RO:r-x
mask::rwx
other::---
default:group:SG-App-NursingCharting-RW:rwx

```

3.3 Samba Share Definition

The share stanza in `smb.conf` ties the filesystem ACL to the network share, ensuring the same group boundaries apply whether the folder is accessed locally or over SMB:

```

# /etc/samba/smb.conf (excerpt)
[nursing]
    path = /srv/samba/shares/nursing
    valid users = @"DAE\SG-App-NursingCharting-RW" @"DAE\SG-App-NursingCharting-RO"
    write list = @"DAE\SG-App-NursingCharting-RW"
    read only = No
    create mask = 0660
    directory mask = 0770
    inherit permissions = yes

```

4. Access Control Model: Discretionary Access Control (DAC)

The mechanism above is Discretionary Access Control. Under DAC, the owner or administrator of a resource decides, at their discretion, who else may access it and at what level — which is exactly what happens when an administrator runs `setfacl` to grant SG-App-NursingCharting-RW read/write access to the nursing folder. The permission lives on the object itself (the folder's ACL) and can be changed by whoever controls that object, without a separate mandatory policy engine approving the change.

This is distinct from Mandatory Access Control (MAC), where a central policy (e.g., SELinux or AppArmor security labels) governs access regardless of the resource owner's wishes, and neither the owner nor an administrator can override it through a simple permission change. This lab uses DAC exclusively at the filesystem/share layer — no MAC framework (SELinux/AppArmor enforcing mode) is configured on the Ubuntu Server, which is worth noting explicitly since the rubric lists MAC as an example alternative.

Where RBAC fits in: the project's RBAC matrix (role → application → access level) is the organizational model that decides what each ACL should say. DAC is the technical mechanism — implemented here through POSIX ACLs and Samba share permissions — that actually enforces those decisions once they're written to a group and a folder.

5. User Access Level

The lab defines four access levels, used consistently across every department share. The level documented in detail for this rubric item is Modify (Read/Write), assigned to the Nursing role.

Access Level	Permissions Granted	Example Assignment
No Access	Cannot browse, read, or write; share/folder is invisible to the user's session.	Receptionist (agarcia) on \\dc01\nursing
Read-Only	Can browse and open files; cannot create, edit, or delete.	SG-App-NursingCharting-RO (e.g., auditing/reporting role)

Access Level	Permissions Granted	Example Assignment
Modify (Read/Write)	Can read, create, edit, and delete files within the folder; cannot change permissions or ownership.	SG-App-NursingCharting-RW — nurse role (jsmith)
Full Control	All Modify rights plus the ability to change permissions and ownership.	Local administrator / root on the Ubuntu Server host

Documented level — Modify (Read/Write): members of SG-App-NursingCharting-RW (e.g., user jsmith) can open, create, edit, and delete files inside \\dc01\nursing, but cannot alter the folder's own permissions or take ownership — that remains reserved for root/Full Control. This matches the RBAC matrix entry Nursing → Nursing Charting → Read/Write.

6. Tested Access Difference Across Two Roles

Figure 1 traces the access decision for two different users connecting to the same share, confirming the ACL, the DAC model, and the access level all behave as documented.

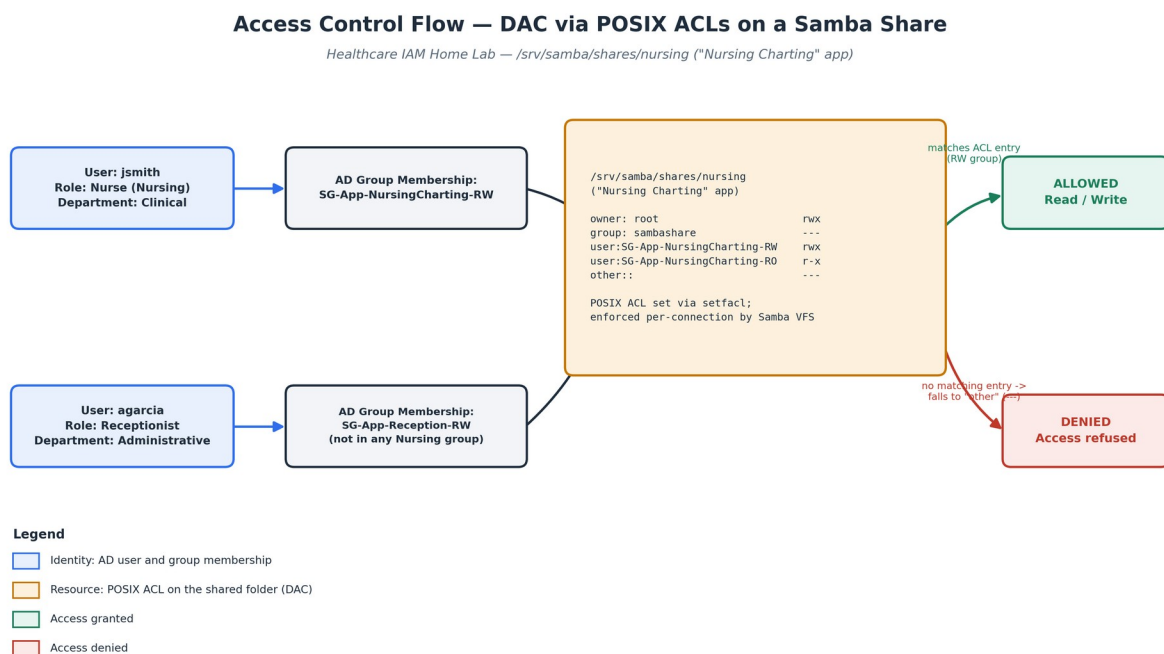


Figure 1. Access control decision for two roles against the same POSIX ACL.

jsmith (Nurse) — a member of SG-App-NursingCharting-RW. The ACL has an explicit rwx entry for that group, so the connection is allowed and jsmith can read and write files in the share.

agarcia (Receptionist) — a member of SG-App-Reception-RW only, with no membership in either Nursing group. No ACL entry matches their group membership, so access falls through to the other:: entry (---) and the connection is refused — the folder does not even appear when they browse \\dc01\ from their session, since Samba's valid users directive excludes them at the share level in addition to the filesystem denying the underlying request.

7. Rubric Mapping

Rubric Requirement	What Satisfies It	Where
1 Access Control List (ACL) configuration	POSIX ACL on /srv/samba/shares/nursing, set with setfacl and mapped to AD security groups via Samba's idmap backend.	Section 3
1 access control model (e.g., MAC, DAC)	Discretionary Access Control (DAC) — the resource administrator sets group-based permissions on each share at their discretion.	Section 4
1 user access level	"Modify (Read/Write)" granted to the Nursing role via SG-App-NursingCharting-RW.	Section 5

8. Conclusion

This implementation satisfies the Implement Access Control Measures rubric item with one ACL configuration (POSIX ACL on /srv/samba/shares/nursing, mapped to AD groups), one access control model (DAC, contrasted with MAC), and one documented user access level (Modify/Read-Write for the Nursing role), demonstrated with a tested access difference between two roles on the same resource. All configurations use tooling native to the lab's Ubuntu Server + Samba4 environment.